25

Displaying Modal Screens

The goal of this chapter is to show you how to present information to the user in ways that don't disrupt the current page. Pages use a `View` component and render it directly on the screen. There are times, however, when there's important information that the user needs to see but you don't necessarily want to kick them off the current page.

You'll start by learning how to display important information. By knowing what information is important and when to use it, you'll learn how to get user acknowledgment: both for error and success scenarios. Then, you'll implement passive notifications that show the user that something has happened. Finally, you'll implement **modal views** that show that something is happening in the background.

The following topics will be covered in this chapter:

- Terminology definitions
- Getting user confirmation

- Passive notifications
- Activity modals

Technical requirements

You can find the code files for this chapter on GitHub at https://github.com/PacktPublishing/React-and-React-Native-5E/tree/main/Chapter25.

Terminology definitions

Before you dive into implementing alerts, notifications, and confirmations, let's take a few minutes and think about what each of these items means. I think this is important because if you end up passively notifying the user about an error, it can easily get missed. Here are my definitions of the types of information that you'll want to display:

- **Alert**: Something important just happened, and you need to ensure that the user sees what's going on. Possibly, the user needs to acknowledge the alert.
- **Confirmation**: This is part of an alert. For example, if the user has just performed an action and then wants to make sure that it was successful before carrying on, they would have to confirm that they've seen the information in order to close the modal. A confirmation can also exist within an

alert, warning the user about an action that they're about to perform.

- **Notification**: Something happened but it's not important enough to completely block what the user is doing. These typically go away on their own.

The trick is to try to use notifications where the information is good to know but not critical. Use confirmations only when the workflow of the feature cannot continue without the user acknowledging what's going on. In the following sections, you'll see examples of alerts and notifications that are used for different purposes.

Getting user confirmation

In this section, you'll learn how to show modal views in order to get confirmation from the user. First, you'll learn how to implement a successful scenario, where an action generates a successful outcome that you want the user to be aware of. Then, you'll learn how to implement an error scenario where something went wrong and you don't want the user to move forward without acknowledging the issue.

Displaying a success confirmation

Let's start by implementing a modal view that's displayed as a result of the user successfully performing an action. Here's the

`Modal` component, which is used to show the user a **confirmation modal**:

```
type Props = ModalProps & {
  onPressConfirm: () => void;
  onPressCancel: () => void;
};
export default function ConfirmationModal({
  onPressConfirm,
  onPressCancel,
  ...modalProps
}: Props) {
  return (
    <Modal transparent onRequestClose={() => {}} {...modalProps}>
      <View style={styles.modalContainer}>
        <View style={styles.modalInner}>
          <Text style={styles.modalText}>Dude, srsly?</Text>
          <Text style={styles.modalButton} onPress={onPressConfirm}>
            Yep
          </Text>
          <Text style={styles.modalButton} onPress={onPressCancel}>
            Nope
          </Text>
        </View>
      </View>
    </Modal>
  );
}
```

The properties that are passed to `ConfirmationModal` are forwarded to the React Native `Modal` component. You'll see why in a moment. First, let's see what this confirmation modal looks like:


12:59
Dude, srsly?
Yep
Nope

Figure 25.1: The confirmation modal

The modal that's displayed once the user completes an action uses our own styling and confirmation message. It also has two actions, but it may only need one, depending on whether this confirmation is pre-action or post-action. Here are the styles that are being used for this modal:

```
  modalContainer: {
    flex: 1,
    justifyContent: "center",
    alignItems: "center",
  },
  modalInner: {
    backgroundColor: "azure",
    padding: 20,
    borderWidth: 1,
    borderColor: "lightsteelblue",
    borderRadius: 2,
    alignItems: "center",
  },
  modalText: {
    fontSize: 16,
    margin: 5,
    color: "slategrey",
```

```
  },
  modalButton: {
    fontWeight: "bold",
    margin: 5,
    color: "slategrey",
  },
```

With the React Native `Modal` component, it's pretty much up to you how you want your confirmation modal view to look. Think of them as regular views, with the only difference being that they're rendered on top of other views.

A lot of the time, you might not care to style your own modal views. For example, in web browsers, you can simply call the `alert()` function, which shows text in a window that's styled by the browser. React Native has something similar: `Alert.alert()`. This is how we can open a native alert:

```
function toggleAlert() {
    Alert.alert("", "Failed to do the thing...", [
      {
        text: "Dismiss",
      },
    ]);
  }
```

Here's what the alert looks like on iOS:

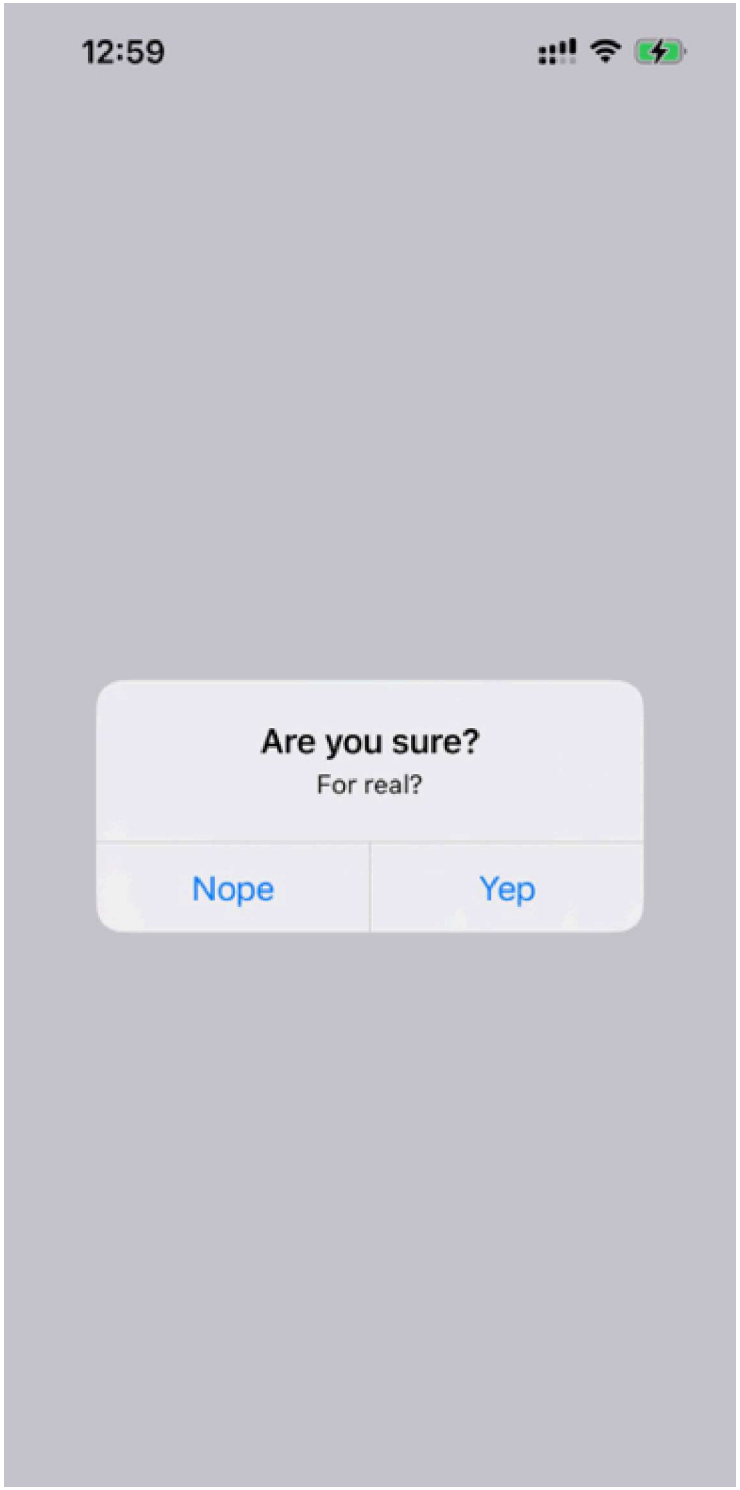

12:59
Are you sure?
For real?
Nope
Yep

Figure 25.2: A confirmation alert on iOS

In terms of functionality, there's nothing really different here. There is a title and text beneath it, but that's something that can easily be added to a modal view if you wanted. The real difference is that this modal looks like an iOS modal instead of something that's styled by the app. Let's see how this alert appears on Android:

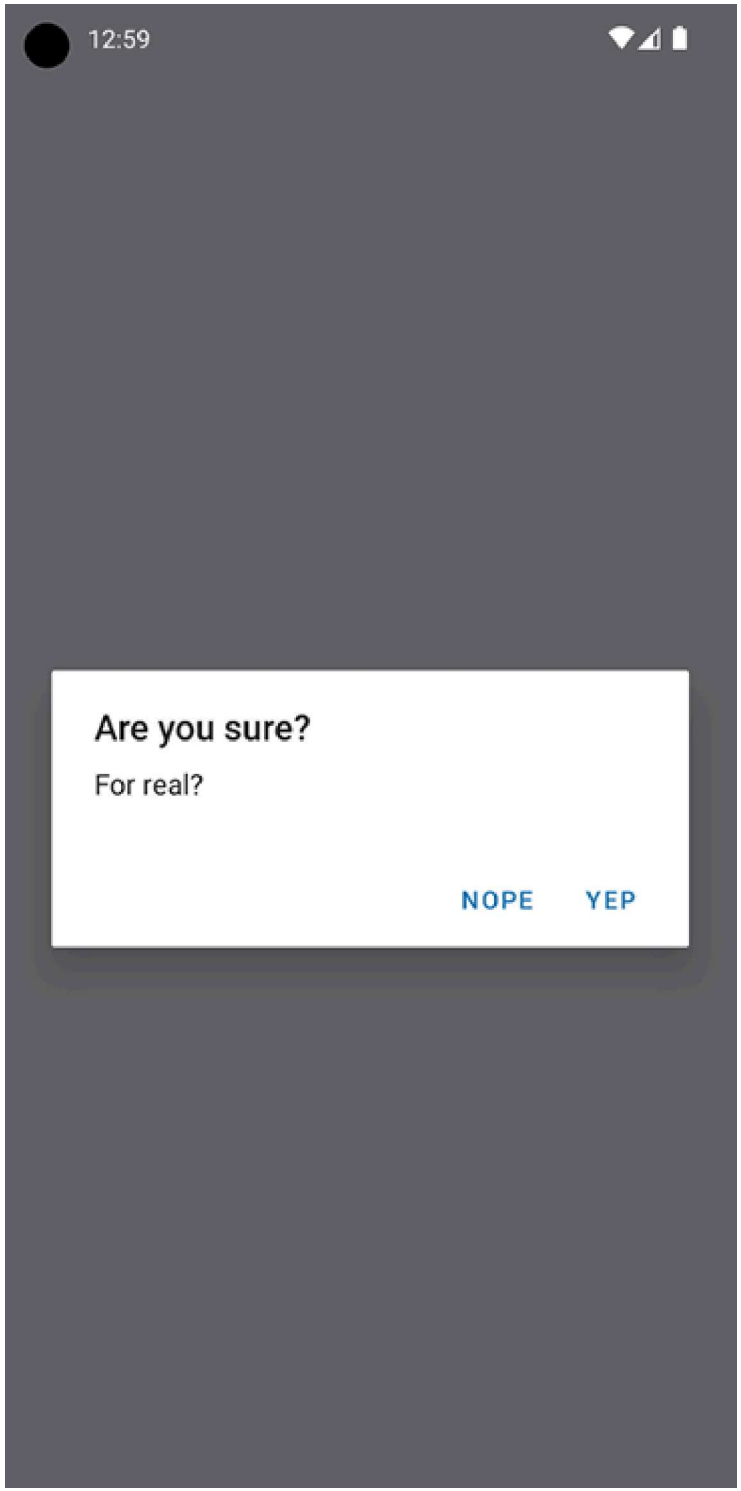

12:59
Are you sure?
For real?
NOPE
YEP

Figure 25.3: A confirmation alert on Android

This modal looks like an Android modal, and you didn't have to style it. I think using alerts over modals is a better choice most of the time. It makes sense to have something styled to look like it's part of iOS or Android. However, there are times when you need more control over how the modal looks, such as when displaying error confirmations.

The approach to rendering modals is different from the approach to rendering alerts. However, they're both still declarative components that change based on the changing property values.

Error confirmation

All of the principles you learned about in the *Displaying a success confirmation* section are applicable when you need the user to acknowledge an error. If you need more control of the display, use a modal. For example, you might want the modal to be red and scary-looking, like this:


13:01
Epic fail!
Fix it
Ignore it

Figure 25.4: The error confirmation modal

Here are the styles that were used to create this look. Maybe you want something a bit more subtle, but the point is that you can make this look however you want:

```
  modalInner: {
    backgroundColor: "azure",
    padding: 20,
    borderWidth: 1,
    borderColor: "lightsteelblue",
    borderRadius: 2,
    alignItems: "center",
  },
```

In the `modalInner` style property, we've defined screen styles. Next, we'll define modal styles:

```
  modalInnerError: {
    backgroundColor: "lightcoral",
    borderColor: "darkred",
  },
  modalText: {
    fontSize: 16,
```

```
    margin: 5,
    color: "slategrey",
  },
  modalTextError: {
    fontSize: 18,
    color: "darkred",
  },
  modalButton: {
    fontWeight: "bold",
    margin: 5,
    color: "slategrey",
  },
  modalButtonError: {
    color: "black",
  },
```

The same modal styles that you used for the success confirmations are still here. That's because the error confirmation modal needs many of the same style properties.

Here's how you apply both to the `Modal` component:

```
const innerViewStyle = [styles.modalInner, styles.modalInnerError];
const textStyle = [styles.modalText, styles.modalTextError];
const buttonStyle = [styles.modalButton, styles.modalButtonError];
type Props = ModalProps & {
  onPressConfirm: () => void;
  onPressCancel: () => void;
};
```

```
export default function ErrorModal({
  onPressConfirm,
  onPressCancel,
  ...modalProps
}: Props) {
  return (
    <Modal transparent onRequestClose={() => {}} {...modalProps}>
      <View style={styles.modalContainer}>
        <View style={innerViewStyle}>
          <Text style={textStyle}>Epic fail!</Text>
          <Text style={buttonStyle} onPress={onPressConfirm}>
            Fix it
          </Text>
          <Text style={buttonStyle} onPress={onPressCancel}>
            Ignore it
          </Text>
        </View>
      </View>
    </Modal>
  );
}
```

The styles are combined as arrays before they're passed to the `style` component property. The styles error always comes last, since conflicting style properties, such as `backgroundColor`, will be overridden by whatever comes later in the array.

In addition to styles in error confirmations, you can include whatever advanced controls you want. It really depends on how your application lets users cope with errors: for example, maybe there are several courses of action that can be taken.

However, the more common case is that something went wrong, and there's nothing you can do about it besides making sure that the user is aware of the situation. In these cases, you can probably get away with just displaying an alert:

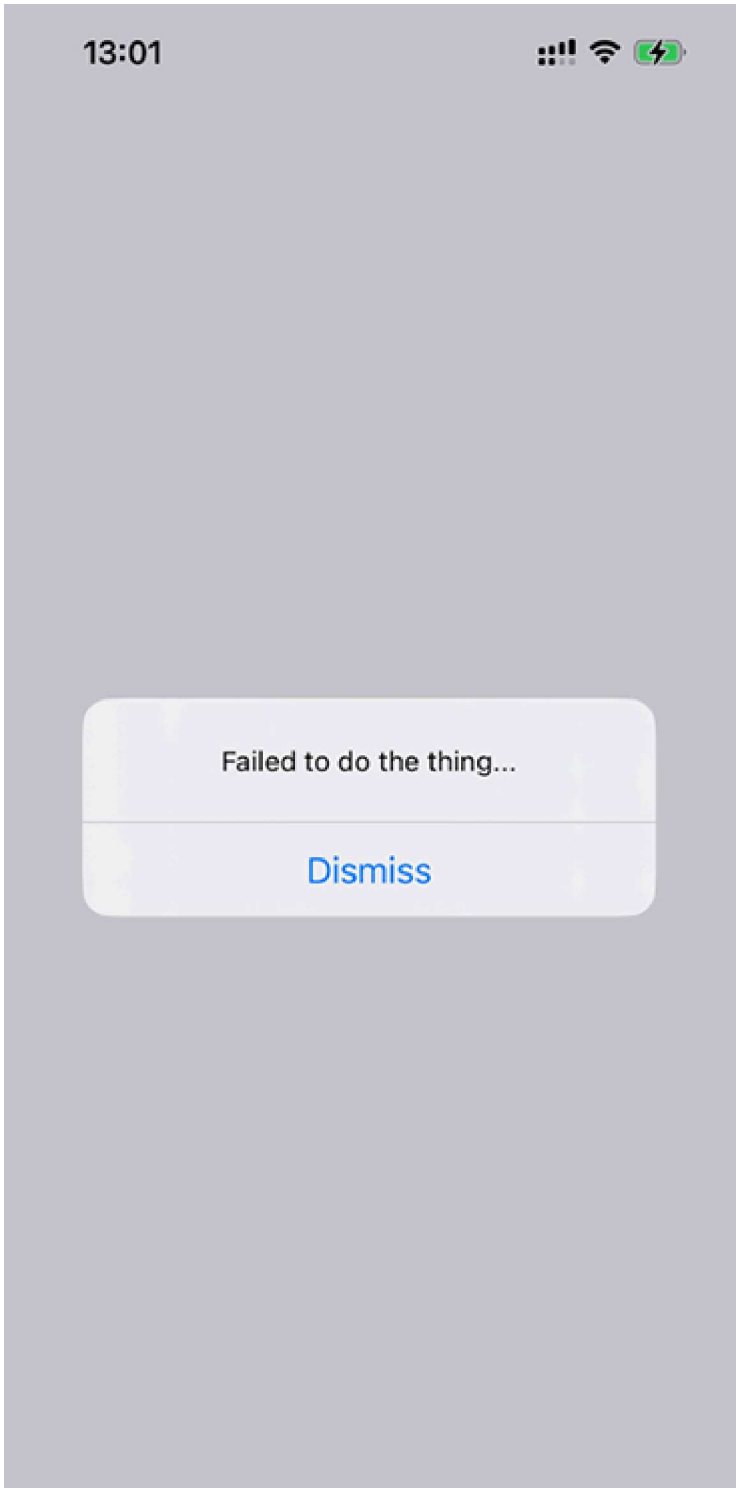

13:01
Failed to do the thing...
Dismiss

Figure 25.5: An error alert

Now that you're able to display error notifications that require user engagement, it's time to learn about less aggressive notifications that don't disrupt what the user is currently doing.

Passive notifications

The notifications you've examined so far in this chapter all have required input from the user. This is by design because it's important information that you're forcing the user to look at. However, you don't want to overdo this. For notifications that are important but not life-altering if ignored, you can use **passive notifications**. These are displayed in a less obtrusive way than modals and don't require any user action to dismiss them.

In this section, you'll create an app that uses the **Toast API** provided by the `react-native-root-toast` library. It's called the Toast API because the information that's displayed looks like a piece of toast popping up. `Toasts` is a common component in Android to show some basic information that does not require user response. Since there is no Toast API for iOS, we will use a

library that implements a similar API that works well on both platforms.

Here's what the `App` component looks like:

```
export default function PassiveNotifications() {
  return (
    <RootSiblingParent>
      <View style={styles.container}>
        <Text
          onPress={() => {
            Toast.show("Something happened!", {
              duration: Toast.durations.LONG,
            });
          }}
        >
          Show Notification
        </Text>
      </View>
    </RootSiblingParent>
  );
}
```

First we should wrap our app in the `RootSiblingParent` component and then we are ready to work with Toast API. To open a toast, we call the `Toast.show` method.

Here's what the Toast notification looks like:

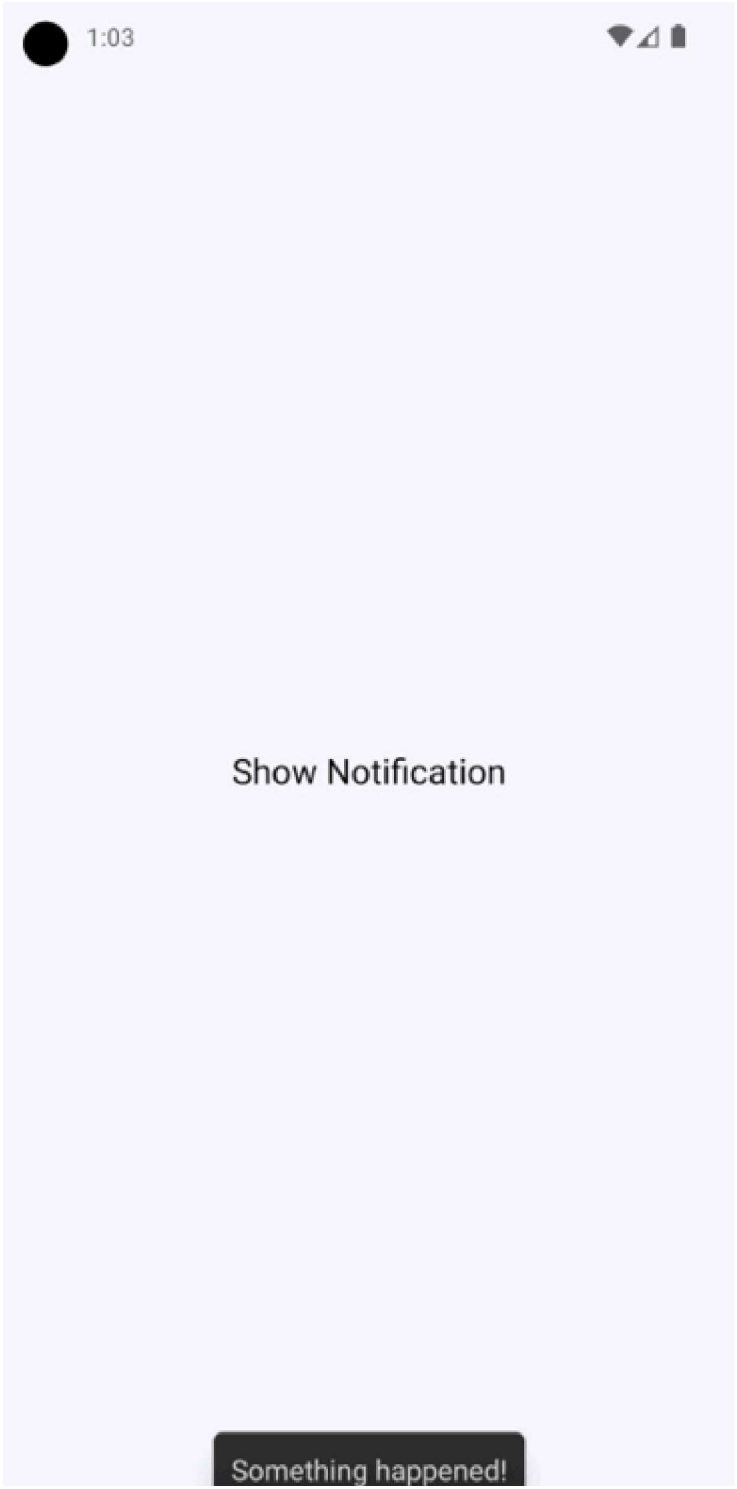
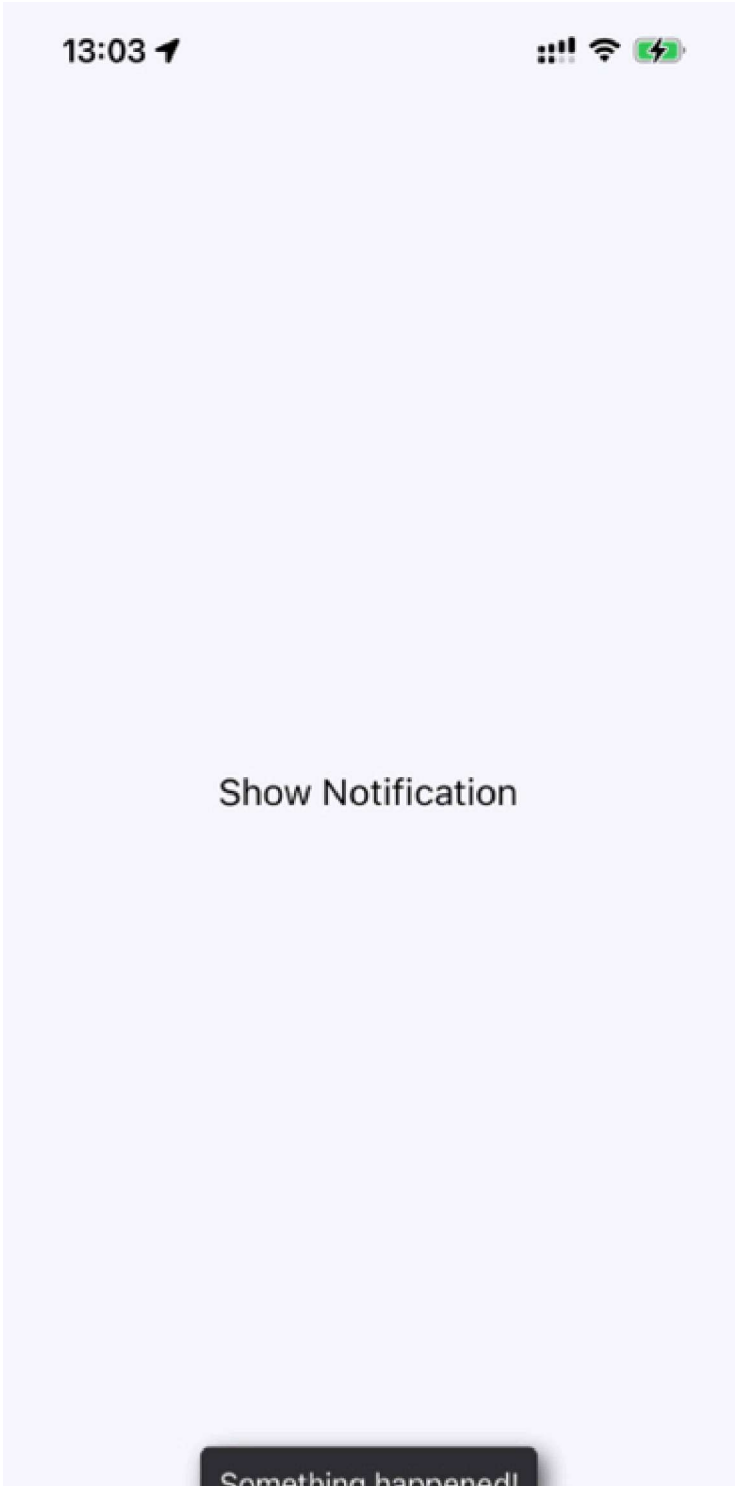

1:03
Show Notification
Something happened!

Figure 25.6: An Android toast

A notification stating **Something happened!** is displayed at the bottom of the screen and is removed after a short delay. The key is that the notification is unobtrusive.

Let’s take a look at how the same toasts look on an iOS device:


13:03
Show Notification
Something happened!

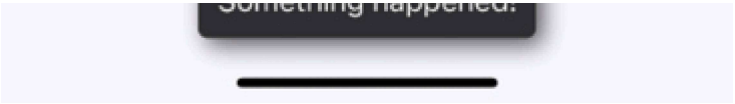


Figure 25.7: An iOS notification

In the next section, you'll learn about activity modals, which show the user that something is happening.

Activity modals

In this final section of this chapter, you'll implement a modal that shows a progress indicator. The idea is to display the modal and then hide it when the promise resolves. Here's the code for the generic `Activity` component, which shows a modal with `ActivityIndicator`:

```
type ActivityProps = {
  visible: boolean;
  size?: "small" | "large";
};
export default function Activity({ visible, size = "large" }: ActivityProps) {
  return (
    <Modal visible={visible} transparent>
      <View style={styles.modalContainer}>
        <ActivityIndicator size={size} />
      </View>
    </Modal>
```

```
    );
  }
```

You might be tempted to pass the promise to the component so that it automatically hides when the promise resolves. I don't think this is a good idea because then you would have to introduce the state into this component. Furthermore, it would depend on a promise in order to function. With the way you've implemented this component, you can show or hide the modal based on the `visible` property alone.

Here's what the activity modal looks like on iOS:


13:06
Fetch Stuff...

Figure 25.8: An activity modal

There's a semi-transparent background on the modal that's placed over the main view with the **Fetch Stuff...** link. By clicking on this link, we will be shown the **activity loader**. Here's how this effect is created in `styles.js`:

```
modalContainer: {
    flex: 1,
    justifyContent: "center",
    alignItems: "center",
    backgroundColor: "rgba(0, 0, 0, 0.2)",
  },
```

Instead of setting the actual `Modal` component to transparent, you can set the transparency in `backgroundColor`, which gives the look of an overlay. Now, let's take a look at the code that controls this component:

```
export default function App() {
  const [fetching, setFetching] = useState(false);
  const [promise, setPromise] = useState(Promise.resolve());
  function onPress() {
    setPromise(
      new Promise((resolve) => setTimeout(resolve, 3000)).then(() => {
        setFetching(false);
```

```
        })
      );
      setFetching(true);
    }
    return (
      <View style={styles.container}>
        <Activity visible={fetching} />
        <Text onPress={onPress}>Fetch Stuff...</Text>
      </View>
    );
  }
```

When the fetch link is pressed, a new promise is created that simulates asynchronous network activity. Then, when the promise resolves, you can change the `fetching` state back to `false` so that the activity dialog is hidden.

Summary

In this chapter, we learned about the need to show important information to mobile users. This sometimes involves explicit feedback from the user, even if that just means acknowledging the message. In other cases, passive notifications work better, since they're less obtrusive than confirmation modals.

There are two tools that we can use to display messages to users: modals and alerts. Modals are more flexible because they're just like regular views. Alerts are good for displaying

plain text, and they take care of styling concerns for us. On Android, we have the `ToastAndroid` interface as well. We saw that it's also possible to do this on iOS, but it just requires more work.

In the next chapter, we'll dig deeper into the gesture response system inside React Native, which makes for a better mobile experience than browsers can provide.